

PLAN INTEGRAL DE PRUEBAS

IPN DEV Inventario

Sistema de Gestión de Inventario — Monolito Modular Laravel + Next.js

Versión 1.0 · 10 de mayo de 2026

Corporación Universitaria Alexander von Humboldt (CUE)

Equipo de Desarrollo IPN

Contenido

1.	Introducción y Alcance
2.	Estrategia de Pruebas
3.	Entorno de Pruebas
4.	Casos de Prueba
4.1	Módulo: Autenticación (AUTH)
4.2	Módulo: Frontend (Next.js)
4.3	Pruebas No Funcionales
5.	Métricas de Aceptación
6.	Cronograma
7.	Roles y Responsabilidades
8.	Gestión de Defectos
9.	Apéndices

1. Introducción y Alcance

1.1 Descripción del Sistema

IPN DEV es un sistema integral de gestión de inventario y logística desarrollado como Proyecto Nuclear 3 de Ingeniería de Software en la CUE. La arquitectura implementa un **monolito modular en Laravel** con separación Controller → Service → Repository → Model, y un frontend en **Next.js 14** con manejo de sesiones y validación de formularios.

1.2 Módulos en Alcance

Implementados y en alcance de QA:

- Módulo de Autenticación (Auth): Login, Logout, Gestión de usuarios, Bloqueo automático, RBAC, Bitácora
- Componentes Frontend: Formulario de login, Manejo de tokens, Navegación básica

Pendientes de implementación [FUERA DE ALCANCE v1.0]:

- Módulo de Inventario (CRUD, stock)
- Lógica FEFO
- Dashboard de Gerencia y Reportes
- Gestión de categorías de productos
- Asignaciones de roles dinámicas

1.3 Objetivos del Plan

- Validación funcional de endpoints de autenticación
- Validación de seguridad (contraseñas, tokens, permisos)
- Validación de reglas de negocio (bloqueo, intentos, roles)
- Validación de auditoría (bitácora de accesos)
- Validación del frontend (formularios, manejo de errores)
- Cobertura de código mínima del 70% en módulos críticos
- Preparación de la infraestructura de pruebas para módulos futuros

2. Estrategia de Pruebas

Unitarias

Stack: PHPUnit + Pest / Jest

Métodos individuales sin dependencias externas. Cobertura objetivo: 80% de lógica de negocio. Ejemplos: AuthService::login(), User::estaBloqueado(), validadores LoginRequest.

Integración

Stack: PHPUnit Feature tests / Pest

Interacción entre capas (Controller → Service → Repository → BD). 100% de endpoints reales. Ejemplos: POST /api/v1/auth/login, GET /api/v1/auth/me, POST /api/v1/auth/logout.

Funcionales

Stack: Postman / Next.js Testing Library

Flujos de usuario completos (E2E light). Ejemplos: Login → token → /me → Logout; 5 intentos fallidos → bloqueo; Admin crea usuario → usuario hace login.

No Funcionales

Stack: Laravel Telescope / DevTools / Apache JMeter

Rendimiento (login < 500 ms), seguridad (RBAC, bcrypt, HTTPS), escalabilidad (100 logins simultáneos), usabilidad (WCAG 2.1 AA).

3. Entorno de Pruebas

3.1 Stack Tecnológico

Backend	PHP 8.3 + Laravel 11	Servidor API
BD Pruebas	SQLite (test) / MySQL 8.0 (staging)	Persistencia
Pruebas API	PHPUnit 11.x + Pest 2.x	Pruebas de integración
Frontend	Next.js 14 + Testing Library	Renderizado + validación
Tests UI	Jest + React Testing Library	Componentes frontend
API Testing	Postman / Insomnia	Validación de endpoints
Performance	Laravel Telescope / DevTools	Monitoreo
CI/CD	GitHub Actions	Automatización

3.2 Datos de Prueba (Seeders)

admin@inventario.test	Admin1234!	administrador	Activo
gerente@inventario.test	Gerente1234!	gerencia	Activo
operario@inventario.test	Operario1234!	encargado_inventarios	Activo

4. Casos de Prueba

4.1 Módulo: Autenticación (AUTH)

Grupo: Login

TC-AUTH-01 Login exitoso con credenciales válidas ALTA	
Módulo	Autenticación
Precondiciones	Usuario admin@inventario.test registrado y activo. Contraseña: Admin1234!
Pasos	1. POST /api/v1/auth/login 2. Body: {email, password} 3. Verificar HTTP 200 4. Validar estructura de respuesta
Resultado esperado	HTTP 200, token Bearer válido, datos del usuario (sin campo password), rol: administrador
Tipo	Integración
Notas	RFAUT01: autenticación básica

TC-AUTH-02 Login fallido — email incorrecto ALTA	
Módulo	Autenticación
Precondiciones	BD sin usuario noexiste@test.com
Pasos	1. POST /api/v1/auth/login 2. Body: email inexistente 3. Verificar respuesta
Resultado esperado	HTTP 401, mensaje genérico 'Credenciales incorrectas', NO revela si el email existe, bitácora: login_fallido user_id=null
Tipo	Integración
Notas	Seguridad RNFSEC: mensajes genéricos

TC-AUTH-03 Login fallido — contraseña incorrecta ALTA	
Módulo	Autenticación
Precondiciones	Usuario admin@inventario.test existe. intentos_fallidos = 0
Pasos	1. POST /api/v1/auth/login con password incorrecto 2. Verificar respuesta 3. Consultar intentos_fallidos en BD
Resultado esperado	HTTP 401, 'Credenciales incorrectas', intentos_fallidos incrementado a 1, bitácora: login_fallido
Tipo	Integración
Notas	RFAUT01: conteo de intentos

TC-AUTH-04 Bloqueo automático tras 5 intentos fallidos		ALTA
Módulo	Autenticación	
Precondiciones	Usuario testeo@test.com activo, intentos_fallidos=0, bloqueado_hasta=null	
Pasos	1. Realizar 5 POST con password incorrecto 2. Verificar 5to intento 3. Consultar bloqueado_hasta en BD 4. Intentar login nuevamente	
Resultado esperado	Intentos 1-4: HTTP 401. Intento 5: HTTP 401 + 'Cuenta bloqueada'. bloqueado_hasta = NOW()+15min. Intento 6: rechazado con mensaje de bloqueo	
Tipo	Integración	
Notas	RFAUT01: seguridad contra fuerza bruta	

TC-AUTH-05 Desbloqueo automático tras expirar bloqueo		MEDIA
Módulo	Autenticación	
Precondiciones	Usuario bloqueado@test.com con bloqueado_hasta = NOW() - 1 min (ya expirado)	
Pasos	1. POST /api/v1/auth/login con credenciales correctas 2. Verificar respuesta 3. Consultar intentos_fallidos y bloqueado_hasta	
Resultado esperado	HTTP 200, token generado, intentos_fallidos reseteado a 0, bloqueado_hasta = null	
Tipo	Integración	
Notas	Recuperación automática de bloqueo	

TC-AUTH-06 Usuario inactivo no puede hacer login		ALTA
Módulo	Autenticación	
Precondiciones	Usuario inactivo@test.com con activo=false, contraseña correcta	
Pasos	1. POST /api/v1/auth/login con credenciales válidas 2. Verificar respuesta	
Resultado esperado	HTTP 403 Forbidden, 'Usuario inactivo. Contacte al administrador.'	
Tipo	Integración	
Notas	RFAUT04: gestión de usuarios activos/inactivos	

TC-AUTH-07 Validación de formato — email inválido		MEDIA
Módulo	Autenticación	
Precondiciones	Ninguna	
Pasos	1. POST /api/v1/auth/login 2. Body: {email: 'notanemail', password: 'Test1234!'}	
Resultado esperado	HTTP 422, error: 'El correo electrónico no tiene un formato válido.'	
Tipo	Integración	
Notas	Validación FormRequest en backend	

TC-AUTH-08 Validación — contraseña muy corta		MEDIA
Módulo	Autenticación	
Precondiciones	Ninguna	
Pasos	1. POST /api/v1/auth/login 2. Body: {email: 'admin@test.com', password: '123'}	
Resultado esperado	HTTP 422, error: 'La contraseña debe tener al menos 6 caracteres.'	
Tipo	Integración	

TC-AUTH-09 Validación — falta el campo email		BAJA
Módulo	Autenticación	
Precondiciones	Ninguna	
Pasos	1. POST /api/v1/auth/login 2. Body: {password: 'Test1234!'} (sin email)	
Resultado esperado	HTTP 422, error: 'El correo electrónico es obligatorio.'	
Tipo	Integración	

Grupo: Logout

TC-AUTH-10 Logout exitoso — revoca token actual		ALTA
Módulo	Autenticación	
Precondiciones	Usuario autenticado con token válido	
Pasos	1. GET /api/v1/auth/me (verificar funciona) 2. POST /api/v1/auth/logout 3. Verificar respuesta 4. GET /api/v1/auth/me con token revocado	
Resultado esperado	Logout: HTTP 200. Segundo GET /me: HTTP 401. Bitácora: logout registrado. Token eliminado de personal_access_tokens	
Tipo	Integración	
Notas	RFAUT03: revocación de sesión	

TC-AUTH-1 1 Logout sin autenticación — debe rechazarse		MEDIA
Módulo	Autenticación	
Precondiciones	Sin token	
Pasos	1. POST /api/v1/auth/logout sin header Authorization	
Resultado esperado	HTTP 401 Unauthorized	
Tipo	Integración	

Grupo: Obtener usuario autenticado (/me)

TC-AUTH-1 2 GET /me con token válido		ALTA
Módulo	Autenticación	
Precondiciones	Token válido de admin@inventario.test (rol: administrador)	
Pasos	1. GET /api/v1/auth/me con header Authorization: Bearer {token} 2. Verificar respuesta	
Resultado esperado	HTTP 200, estructura: success, data.id, data.name, data.email, data.role.nombre. Campo password NO aparece	
Tipo	Integración	
Notas	RFAUT02: información del usuario	

TC-AUTH-1 3 GET /me sin token — debe rechazarse		MEDIA
Módulo	Autenticación	
Precondiciones	Sin token	
Pasos	1. GET /api/v1/auth/me sin Authorization	
Resultado esperado	HTTP 401 Unauthorized	
Tipo	Integración	

TC-AUTH-1 4 GET /me con token expirado (> 30 min)		MEDIA
Módulo	Autenticación	
Precondiciones	Token generado hace 35 min. Configuración: expiración 30 min	
Pasos	1. GET /api/v1/auth/me con token antiguo	
Resultado esperado	HTTP 401 Unauthorized	
Tipo	Integración	
Notas	[PENDIENTE: Confirmar config de expiración en Sanctum]	

Grupo: Crear usuario (solo Admin)

TC-AUTH-1 5		Crear usuario exitoso como administrador	ALTA
Módulo	Autenticación		
Precondiciones	Token de administrador válido. Email nuevo@test.com no existe		
Pasos	1. POST /api/v1/auth/usuarios 2. Headers: Authorization: Bearer {admin_token} 3. Body: name, email, password, password_confirmation, role_id=2		
Resultado esperado	HTTP 201 Created, success=true, data.id/name/email. Usuario en BD con activo=true. Contraseña hasheada (nunca plaintext)		
Tipo	Integración		
Notas	RFAUT02: solo admin puede crear usuarios		

TC-AUTH-1 6		Crear usuario — email duplicado rechazado	ALTA
Módulo	Autenticación		
Precondiciones	Token admin. Email admin@inventario.test ya existe		
Pasos	1. POST /api/v1/auth/usuarios con email existente		
Resultado esperado	HTTP 422, 'Ya existe un usuario registrado con ese correo electrónico.'		
Tipo	Integración		

TC-AUTH-1 7		Crear usuario — contraseña no coincide	MEDIA
Módulo	Autenticación		
Precondiciones	Token de administrador		
Pasos	1. POST con password='Pass1234!' y password_confirmation='DifferentPass!'		
Resultado esperado	HTTP 422, 'La confirmación de la contraseña no coincide.'		
Tipo	Integración		

TC-AUTH-1 8		Crear usuario — contraseña muy corta	MEDIA
Módulo	Autenticación		
Precondiciones	Token de administrador		
Pasos	1. POST con password='Short1!' (menos de 8 caracteres)		
Resultado esperado	HTTP 422, 'La contraseña debe tener al menos 8 caracteres.'		
Tipo	Integración		

TC-AUTH-1 9 Crear usuario — rol inexistente MEDIA	
Módulo	Autenticación
Precondiciones	Token de administrador
Pasos	1. POST con role_id=9999 (no existe en BD)
Resultado esperado	HTTP 422, 'El rol seleccionado no existe en el sistema.'
Tipo	Integración

TC-AUTH-2 0 Crear usuario sin permiso — usuario con rol gerencia ALTA	
Módulo	Autenticación
Precondiciones	Token de usuario con rol: gerencia (NO administrador)
Pasos	1. POST /api/v1/auth/usuarios con token de gerencia 2. Body con datos válidos
Resultado esperado	HTTP 403 Forbidden, 'No tienes permiso para esta acción'
Tipo	Integración
Notas	RFAUT02: control de acceso por rol

Grupo: Auditoría (Bitácora de Accesos)

TC-AUTH-2 1 Bitácora registra login exitoso ALTA	
Módulo	Autenticación
Precondiciones	Tabla bitacora_accesos existente. Usuario: admin@inventario.test
Pasos	1. Contar registros actuales 2. POST /api/v1/auth/login exitoso 3. Contar registros nuevamente 4. Verificar último registro
Resultado esperado	Nuevo registro: user_id, accion='login_exitoso', ip_address (IPv4 válida), user_agent, created_at
Tipo	Integración
Notas	RNF-MAN: trazabilidad de accesos

TC-AUTH-2 2 Bitácora registra login fallido (email no existe) MEDIA	
Módulo	Autenticación
Precondiciones	Email noexiste@test.com no registrado
Pasos	1. POST con email inexistente 2. Verificar último registro en bitacora_accesos
Resultado esperado	Registro con user_id=NULL, accion='login_fallido', ip_address y user_agent capturados
Tipo	Integración

TC-AUTH-23		Bitácora registra logout	MEDIA
Módulo	Autenticación		
Precondiciones	Token válido de usuario autenticado		
Pasos	1. POST /api/v1/auth/logout 2. Verificar último registro en bitacora_accesos		
Resultado esperado	Registro con accion='logout', user_id = id del usuario que hizo logout		
Tipo	Integración		

4.2 Módulo: Frontend (Next.js)

TC-FRONT-01		Renderizado del formulario de login	MEDIA
Módulo	Frontend		
Precondiciones	Frontend en http://localhost:3000		
Pasos	1. Navegar a /login 2. Inspeccionar elementos 3. Verificar presencia de inputs y botón		
Resultado esperado	Input email con label 'Correo electrónico', input password con toggle mostrar/ocultar, botón submit visible y clickeable		
Tipo	Funcional		

TC-FRONT-02		Envío de credenciales y guardado de token	ALTA
Módulo	Frontend		
Precondiciones	Backend en http://localhost:8000. Usuario: admin@inventario.test / Admin1234!		
Pasos	1. Llenar email y password 2. Click en 'Iniciar sesión' 3. Inspeccionar localStorage 4. Verificar redirección		
Resultado esperado	POST enviado a /api/v1/auth/login, token guardado en localStorage, redirección a /dashboard		
Tipo	Funcional		

TC-FRONT-03		Manejo de error de credenciales inválidas	ALTA
Módulo	Frontend		
Precondiciones	Email y/o password incorrectos		
Pasos	1. Ingresar credenciales incorrectas 2. Submit 3. Esperar respuesta		
Resultado esperado	Mensaje de error visible: 'Credenciales incorrectas'. Token NO guardado. NO redirige a /dashboard		
Tipo	Funcional		

TC-FRONT-04		Validación frontend — email vacío	BAJA
Módulo	Frontend		
Precondiciones	Formulario visible		
Pasos	1. Dejar email vacío 2. Llenar password 3. Intentar submit		
Resultado esperado	Validación nativa o custom visible. Petición NO se envía al backend		
Tipo	Funcional		

TC-FRONT-05		Redireccionamiento automático de / a /login	MEDIA
Módulo	Frontend		
Precondiciones	Ninguna		
Pasos	1. Navegar a http://localhost:3000/ 2. Verificar redirección		
Resultado esperado	Redirección automática a /login. URL final: http://localhost:3000/login		
Tipo	Funcional		

4.3 Pruebas No Funcionales

TC-PERF-01 Tiempo de respuesta de login < 500 ms MEDIA	
Módulo	Rendimiento
Precondiciones	BD en localhost o red local. Usuario válido existente
Pasos	1. Medir tiempo de POST /api/v1/auth/login 2. Repetir 10 veces 3. Calcular promedio y P95
Resultado esperado	Promedio < 500 ms. P95 < 700 ms
Tipo	No Funcional (Rendimiento)

TC-SEC-01 Contraseña nunca en logs ni respuestas ALTA	
Módulo	Seguridad
Precondiciones	Login exitoso. Logs habilitados en servidor
Pasos	1. POST /api/v1/auth/login con password: AdminPassword123 2. Revisar respuesta JSON 3. Revisar logs del servidor 4. Revisar Network tab del navegador
Resultado esperado	Respuesta JSON: NO contiene campo password. Logs: NO contienen password en plaintext. HTTPS en producción
Tipo	No Funcional (Seguridad)
Notas	RNFSEC-01

TC-SEC-02 Bcrypt hash de contraseñas verificable ALTA	
Módulo	Seguridad
Precondiciones	Usuario creado o migración ejecutada
Pasos	1. SELECT password FROM users LIMIT 1 2. Verificar formato bcrypt 3. Intentar descryptar (debe fallar) 4. Usar Hash::check() en código
Resultado esperado	Hash comienza con \$2y\$ (bcrypt). Longitud > 50 caracteres. Hash::check() retorna true solo con plaintext correcto
Tipo	No Funcional (Seguridad)

TC-SEC-03 RBAC — middleware role rechaza usuarios sin permiso ALTA	
Módulo	Seguridad
Precondiciones	Rutas protegidas por middleware('role:administrador'). Tokens de usuarios con diferentes roles
Pasos	1. POST /api/v1/auth/usuarios con token de gerencia 2. Verificar respuesta 3. POST con token de admin 4. Verificar respuesta
Resultado esperado	Token gerencia: HTTP 403 Forbidden. Token admin: HTTP 201 Created (con datos válidos)
Tipo	No Funcional (Seguridad)
Notas	RFAUT02

5. Métricas de Aceptación

5.1 Cobertura de Código

Cobertura de líneas (Backend)	≥ 70% en módulo Auth
Cobertura de funciones	≥ 80% en AuthService, UserRepository
Métodos no cubiertos	< 5 en módulos críticos

5.2 Tasa de Éxito

Pruebas unitarias	100% paso
Pruebas de integración	≥ 95% paso
Pruebas E2E	≥ 90% paso

5.3 Tiempo de Ejecución de Suites

Unitarias (Backend)	< 5 segundos
Integración (Backend)	< 30 segundos
Unitarias (Frontend)	< 10 segundos
Suite completa	< 60 segundos

5.4 Defectos

Defectos críticos	0 antes de release
Defectos altos	≤ 2 antes de release
Defectos medios	≤ 5 antes de release

6. Cronograma

Sprint 1 (10–21 May 2026)

Objetivo: Preparar infraestructura y validar módulo Auth — Login

Configurar PHPUnit + Pest	1 día	QA	phpunit.xml listo
Configurar Jest Frontend	1 día	QA	jest.config.js listo
Crear seeders de prueba	1 día	Dev	UserSeeder actualizado
TC-AUTH-01 a TC-AUTH-09 (Login)	2 días	QA	9 tests pasando
TC-AUTH-10 a TC-AUTH-14 (Logout/Me)	1.5 días	QA	5 tests pasando

Sprint 2 (24 May – 4 Jun 2026)

Objetivo: Completar Auth, auditoría y frontend

TC-AUTH-15 a TC-AUTH-20 (Create User)	2 días	QA	6 tests pasando
TC-AUTH-21 a TC-AUTH-23 (Bitácora)	1 día	QA	3 tests pasando
TC-FRONT-01 a TC-FRONT-05	1.5 días	QA	5 tests pasando
TC-PERF-01 (Rendimiento)	0.5 día	QA	Reporte de rendimiento
TC-SEC-01 a TC-SEC-03 (Seguridad)	1 día	QA	3 tests pasando
Reporte de cobertura	1 día	QA	coverage.html

Sprint 3 (7–18 Jun 2026)

Objetivo: Infraestructura para módulos futuros

Refactor de tests (si aplica)	1 día	QA	Code review completado
Documentación de procesos QA	1 día	QA	Wiki/README QA
Configuración CI/CD GitHub Actions	1.5 días	DevOps	Workflows listos
Capacitación de equipo	1 día	QA Lead	Sesión de training

7. Roles y Responsabilidades

7.1 Equipo de QA

QA Lead	Coordinar plan, reportes, decisiones de release
QA Engineer (Backend)	Pruebas API, integración, seguridad
QA Engineer (Frontend)	Pruebas UI, E2E, navegadores
QA Automation	Configurar PHPUnit, Jest, CI/CD

7.2 Equipo de Desarrollo

Backend Dev	Implementación de código según requisitos
Frontend Dev	Componentes, validación HTML5
DevOps	Configuración de entornos y CI/CD

8. Gestión de Defectos

8.1 Clasificación de Severidad

CRÍTICA	Bloquea releases, falla de seguridad, pérdida de datos	Login no funciona, SQL injection	4 horas
ALTA	Funcionalidad principal afectada, error de negocio	RBAC no funciona, token no se revoca	24 horas
MEDIA	Funcionalidad secundaria afectada, UX degradada	Mensaje de error incorrecto, performance < requerida	48 horas
BAJA	Cosméticos, sugerencias	Typo en UI, color incorrecto	1 semana

8.2 Ciclo de Vida del Defecto

ABIERTO → EN REVISIÓN → EN DESARROLLO → EN PRUEBA → CERRADO

8.3 Herramientas

- Reporte: GitHub Issues + Labels (crítica, alta, media, baja)
- Tracking: GitHub Projects (Kanban)
- Comunicación: Slack / Discord para alertas
- Logs: Laravel Telescope (Backend), DevTools (Frontend)

9. Apéndices

A. Matriz de Trazabilidad RF → Casos de Prueba

RFAUT01	Autenticación con email/password	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03
RFAUT01	Bloqueo tras 5 intentos fallidos	TC-AUTH-04, TC-AUTH-05
RFAUT02	RBAC (4 roles)	TC-AUTH-20, TC-SEC-03
RFAUT03	Revocación de sesión en logout	TC-AUTH-10
RFAUT04	Usuarios inactivos no pueden login	TC-AUTH-06
RNFSEC-01	Contraseña hasheada, nunca en plaintext	TC-SEC-01, TC-SEC-02
RNF-MAN	Auditoría con trazabilidad	TC-AUTH-21, TC-AUTH-22, TC-AUTH-23
Tokens	Sanctum con expiración 30 min	TC-AUTH-14

B. Comandos de Ejecución

Backend (PHPUnit + Pest):

```
php artisan test # Todas las pruebas
php artisan test --filter=Auth # Solo módulo Auth
php artisan test --coverage-html # Con cobertura HTML
./vendor/bin/pest --filter=login # Usando Pest
```

Frontend (Jest):

```
npm test # Todas las pruebas
npm test -- --watch # Modo watch
npm test -- --coverage # Con cobertura
npm test LoginForm.test.tsx # Prueba específica
```

C. Pipeline CI/CD (GitHub Actions)

```
name: Tests
on: [push, pull_request]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: shivammathur/setup-php@v2
        with: {php-version: '8.3'}
      - run: cd backend && composer install
      - run: cd backend && php artisan test --coverage
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with: {node-version: '18'}
      - run: cd frontend && npm install
      - run: cd frontend && npm test -- --coverage
```

Historial de Cambios

1.0	2026-05-10	Documento inicial — Módulo Auth completo

Documento preparado por el Equipo de Desarrollo IPN · Proyecto Nuclear 3 — I-2026 · CUE